Mixture-of-Collaboration: Expert Collaboration via Learned Mediators in Sparse Transformer Architectures

Technical Report

Francisco Antonio
Auren Research, Independent

March 2026

Abstract

Mixture-of-Experts (MoE) Transformers scale parameter count while keeping per-token compute bounded, but standard top-k routing still treats selected experts as conditionally independent paths whose outputs are merged only at the end. This report presents *Mixture-of-Collaboration* (MoC), a sparse decoder-only Transformer that augments MoE with explicit expert interaction through a learned mediator. MoC combines three mechanisms: (1) mediator-based collaboration over routed experts with $\mathcal{O}(K)$ token complexity, avoiding pairwise $\mathcal{O}(E^2)$ expert communication; (2) Iterative Reasoning Layers (IRL), where each expert performs weight-shared multi-step refinement; and (3) adaptive compute gates that learn, per token, how many reasoning steps and collaboration rounds to allocate.

We validate MoC at approximately 64M parameters in a controlled four-way ablation: dense baseline, standard MoE, fixed-depth MoC, and adaptive MoC-vNext. Under matched data and training budget, MoC-vNext reaches validation perplexity 59.97 versus 62.89 for standard MoE at comparable total and active parameters, a 4.6% relative improvement. Decomposition shows that sparse routing explains most of the gain, while mediator collaboration and adaptive compute provide additional improvements. We report routing health, adaptive compute usage, and optimization details for reproducibility, and we explicitly document current scale limits: single seed, single dataset, and small-model regime. Code, exact configs, and run logs are publicly available.

1 Introduction

Sparse Transformers based on MoE routing can substantially increase total parameter capacity without making every token pay full dense-compute cost [4, 6, 9, 10]. In standard top-k MoE, a router selects experts and the selected experts process token states independently before a weighted merge. This design is efficient, but it also constrains information exchange among experts to a single late aggregation point.

An immediate idea is to let experts communicate directly. However, many communication-heavy variants introduce pairwise interactions whose cost scales with the number of experts, making them difficult to scale at high expert counts. MoC instead adopts a learned mediator hub: experts communicate through a shared latent state, so per-token collaboration remains linear in selected experts ($\mathcal{O}(K)$), not quadratic in total experts.

A second issue is compute allocation. Fixed-depth processing assumes all tokens require equal effort, but tokens in language modeling vary widely in difficulty. A uniform budget may over-compute

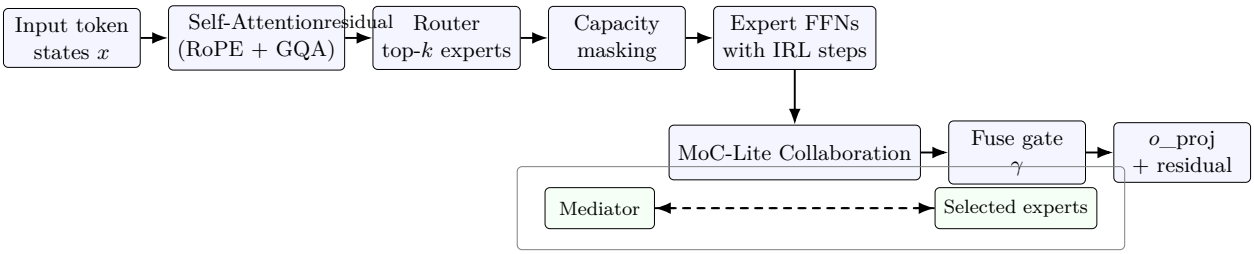

Figure 1: MoC block flow. Tokens are routed to top-k experts, filtered by capacity, processed with IRL, then refined by mediator-based collaboration before gated fusion and output projection.

trivial tokens and under-compute difficult ones. MoC therefore incorporates adaptive gates for both intra-expert reasoning depth and inter-expert collaboration rounds.

Contributions.

- We introduce **MoC**, a sparse MoE architecture with mediator-based expert collaboration at linear cost in top-k experts.
- We integrate **Iterative Reasoning Layers (IRL)** inside experts via weight-shared multi-step refinement.
- We add **adaptive compute gating** for both reasoning depth and collaboration rounds.
- We provide a **controlled ablation** showing measurable gains from routing, collaboration+IRL, and adaptive compute under matched budget.

This document is a technical report (not a peer-reviewed paper) intended to record architecture details and preliminary validation at small scale with reproducible settings.

2 Architecture

2.1 Overview

The implementation in `model_moc.py` is a decoder-only Transformer with Rotary Position Embeddings (RoPE), grouped-query attention (GQA), RMSNorm, and SwiGLU-style FFNs. Each transformer block applies:

$$\textit{Attention} \rightarrow \textit{Router} \rightarrow \textit{Expert Dispatch} \rightarrow \textit{MoC Collaboration} \rightarrow \textit{Fuse/Project} \rightarrow \textit{Residual}.$$

When `n_experts` is `None`, blocks use a dense FFN path. Otherwise, each block uses `MoCTopKExperts` for routing, capacity filtering, per-expert IRL computation, and optional collaboration/fusion.

2.2 Router and Expert Dispatch

Given flattened token states $x \in \mathbb{R}^{N \times d}$, routing logits are computed in FP32:

$$\ell = xW_{\text{gate}}^{\top}, \qquad p = \text{softmax}(\ell). \tag{1}$$

During training, Gaussian router noise (std 0.02 in experiments) perturbs logits for exploration. Top-k experts are selected per token and top-k probabilities are normalized for weighted aggregation.

Capacity is enforced via a vectorized priority-based keep/drop policy:

- flatten all token-expert slots ($N \times K$),
- stable sort by descending routing priority,
- regroup by expert,
- keep up to per-expert capacity, drop the rest.

The code sets capacity from

$$C = 2^{\lceil \log_2(\lceil \text{capacity_factor} \cdot (NK/E) \rceil) \rceil}, \tag{2}$$

clipped to at most NK. Experiments use `capacity_factor=1.5`.

Auxiliary router regularization is:

$$\mathcal{L}_{\text{aux}} = \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \lambda_z \mathcal{L}_z + \lambda_{\text{drop}} \mathcal{L}_{\text{drop}}, \tag{3}$$

where $\lambda_{\text{bal}} = 0.01$, $\lambda_z = 0.001$, and $\lambda_{\text{drop}} = 0.001$ in MoE/MoC runs. The implementation computes:

$$\mathcal{L}_{\text{bal}} = E \sum_{e=1}^{E} \bar{p}_e \, \bar{f}_e, \tag{4}$$

$$\mathcal{L}_z = \mathbb{E}[\log \sum_e \exp(\ell_e)]^2, \tag{5}$$

$$\mathcal{L}_{\text{drop}} = \text{drop fraction}. \tag{6}$$

2.3 Iterative Reasoning Layers (IRL)

Each expert is a weight-shared iterative FFN:

$$h^{(0)} = x, \qquad h^{(t+1)} = h^{(t)} + \alpha \, \text{FFN}\Big(\text{RMSNorm}(h^{(t)} + x)\Big), \tag{7}$$

for $t = 0, \ldots, N_{\max} - 1$, with $\alpha = 1/\sqrt{N_{\max}}$ for stability. The FFN uses SwiGLU structure and reuses the same parameters at every step.

This recurrent refinement is related to equilibrium-style views of deep computation [2]: although MoC runs a finite number of iterations, the shared-weight formulation encourages fixed-point-like behavior while keeping parameter growth controlled.

2.4 MoC-Lite Collaboration

After expert computation, MoC-Lite refines token representation through a learned mediator state. For each token, let selected expert states be $\{e_i\}_{i=1}^{K}$ and weighted MoE merge be $\tilde{e} = \sum_i w_i e_i$.

Mediator initialization is:

$$m_0 = m_{\text{learned}} + W_{\text{ctx}} x \tag{8}$$

(or a fallback from $\tilde{e}$ if learned mediator is disabled).

One collaboration round computes:

$$q = W_q \,\mathrm{Norm}(m), \tag{9}$$

$$k_i = W_k \,\mathrm{Norm}(e_i), \quad v_i = W_v \,\mathrm{Norm}(e_i), \tag{10}$$

$$a_i = \mathrm{softmax}_i\left(\frac{q^\top k_i}{\sqrt{d}}\right), \tag{11}$$

$$u = \sum_i a_i v_i, \tag{12}$$

$$m' = m + \mathrm{MLP}(\mathrm{Norm}(m) + u). \tag{13}$$

If expert feedback is enabled (enabled in experiments), mediator-to-expert feedback is gated per expert:

$$g_i = \sigma(w_e^\top e_i + w_m^\top m'), \qquad e_i' = e_i + g_i \, W_{m\to e} m'. \tag{14}$$

Final fusion between mediator and weighted expert sum uses an input-conditioned gate:

$$\gamma = \sigma(W_f x + b), \qquad y = \gamma m + (1-\gamma)\sum_i w_i e_i'. \tag{15}$$

This hub topology keeps collaboration complexity linear in selected experts ($\mathcal{O}(K)$ per token) instead of all-expert pairwise interaction.

2.5 Adaptive Compute Gating

MoC includes two independent adaptive gates:

- **Reasoning gate** for IRL depth (0 to $N_{\max}$ steps).
- **Collaboration gate** for mediator rounds (0 to $C_{\max}$ rounds).

Each gate maps token state to logits over $\{0, 1, \ldots, S\}$ and applies temperature-scaled softmax. In `soft` mode (used in experiments), activity weights are cumulative tail probabilities, enabling compile-friendly dense execution:

$$\mathrm{activity}_s = \sum_{j=s}^{S} P(\mathrm{steps} = j). \tag{16}$$

In `hard` mode, the model selects an argmax step count and executes sparse per-token control flow. Expected-step penalties encourage efficiency:

$$\mathcal{L}_{\mathrm{step}} = \lambda_{\mathrm{step}} \frac{\mathbb{E}[\mathrm{steps}]}{S}, \tag{17}$$

with weight 0.001 for both reasoning and collaboration in MoC-vNext. Gate temperature is 1.0, gate noise std is 0.02.

Table 1: Controlled experiment configurations (exact settings used).

Model	Expert routing	IRL setup	Collaboration	Adaptive compute	Total params	Active/token	Batch config
`a-vanilla`	none (dense FFN)	$N = 1$, adaptive off	off	off	35,789,952	35,789,952	bs=8, grad_acc=2
`b-moe`	$E = 4$, top-2, cap=1.5	$N = 1$, adaptive off	off (independent experts + weighted merge)	off	64,138,398	49,978,014	bs=4, grad_acc=4
`c-moc-v1`	$E = 4$, top-2, cap=1.5	$N = 2$ fixed	mediator on, feedback on, 2 fixed rounds	collaboration adaptive off	64,131,468	49,971,084	bs=8, grad_acc=2
`d-moc-vnext`	$E = 4$, top-2, cap=1.5	$N \leq 3$, adaptive on (soft)	mediator on, feedback on, up to 2 rounds	reasoning + collaboration adaptive on	64,175,358	49,996,494	bs=8, grad_acc=2

3 Experimental Setup

3.1 Model Configurations

All experiments share the same base decoder architecture:

- `d_model=384`, `n_layers=6`, `n_heads=6`, `n_kv_heads=6`
- `vocab_size=65536`, `max_seq_len=512`
- `ffn_hidden_multiplier=4.0`, `multiple_of=64`
- `dropout=0.0`, `rope_theta=10000.0`

Table 1 lists the four controlled variants.

All MoE/MoC runs share:

- `aux_loss_weight=0.01`
- `router_z_loss_weight=0.001`
- `router_noise_std=0.02`
- `drop_penalty_weight=0.001`

MoC-vNext additionally uses `adaptive_compute_mode=s̈oft¨`, reasoning/collaboration gate temperature 1.0, gate noise std 0.02, and step penalty weights 0.001.

Batch-size note. The config table reflects RX 6600-compatible settings (effective batch size 16 for all runs). Primary reported results were obtained on an NVIDIA A10 with effective batch size 32 (doubled), yielding approximately 328M training tokens: $32 \times 512 \times 20000$.

3.2 Training Setup

- **Max steps:** 20,000
- **Sequence length:** 512
- **Tokens seen (A10, effective batch 32):** $\sim$328M
- **Optimizer:** AdamW, lr 3×10^{-4}, betas $(0.9, 0.95)$, weight decay 0.1
- **Router learning rate:** 0.5$\times$ base lr (set in `configure_optimizers`)
- **Schedule:** 500-step linear warmup then cosine decay to 1% of base lr
- **Precision:** bf16 mixed precision

Table 2: Main validation results (exact values).

Model	Total Params	Active/Token	Val CE	Val PPL	ΔPPL vs Vanilla
`a-vanilla`	35,789,952	35,789,952	4.280	72.24	—
`b-moe`	64,138,398	49,978,014	4.141	62.89	-12.9%
`c-moc-v1`	64,131,468	49,971,084	4.099	60.28	-16.6%
`d-moc-vnext`	64,175,358	49,996,494	4.094	59.97	-17.0%

- **Gradient clipping:** max norm 1.0
- **Gradient checkpointing:** disabled
- **torch.compile:** disabled
- **Seed:** 1337 (fixed)

3.3 Hardware

- NVIDIA A10 24GB (primary runs, reported numbers)
- AMD RX 6600 8GB (secondary validation)
- Single GPU, no distributed training

3.4 Evaluation

Evaluation metrics and cadence:

- validation CE loss (excluding aux loss)
- perplexity exp(validation CE)
- validation every 2,000 steps over 20 batches
- routing diagnostics: expert utilization, drop rate, router entropy
- adaptive metrics: mean reasoning steps, mean collaboration rounds, compute saved percentage

4 Results

4.1 Main Results

Table 2 reports the primary comparison.

Compared to standard MoE, MoC-vNext improves perplexity from 62.89 to 59.97 (4.6% relative). This comparison holds at effectively matched active parameters per token and shared training budget.

Table 3: Supplementary metrics from validation/training logs (exact values).

Metric	a-vanilla	b-moe	c-moc-v1	d-moc-vnext
`val/val_loss`	4.2147	4.0933	4.0298	4.0370
`val/val_aux_loss`	0	0.01023	0.01031	0.01132
`loss/total`	4.2800	4.1517	4.1092	4.1051
`loss/ce`	4.2800	4.1414	4.0989	4.0939

Table 4: Incremental component ablation.

Step	Added component(s)	PPL	Incremental Δ
1	Dense baseline (single FFN, no routing)	72.24	—
2	Add top-2 expert routing ($E = 4$), independent experts + weighted merge	62.89	-12.9%
3	Add mediator collaboration (2 rounds) + IRL (2 fixed steps)	60.28	-4.1%
4	Add adaptive compute (IRL up to 3, adaptive collaboration)	59.97	-0.5%

4.2 Ablation Analysis

We decompose improvements incrementally (Table 4).

Important metric interpretation. `c-moc-v1` has slightly lower total `val/val_loss` (4.0298) than `d-moc-vnext` (4.0370), but this metric includes auxiliary terms. MoC-vNext carries higher aux loss (0.01132 vs 0.01031), likely due to additional adaptive gating dynamics, while still achieving better pure CE (4.094 vs 4.099), which is the core modeling objective.

4.3 Adaptive Compute Behavior

MoC-vNext learns non-uniform compute allocation:

- average IRL reasoning depth: $\sim$2.54 / 3
- average collaboration rounds: $\sim$0.47 / 2
- estimated adaptive compute savings: $\sim$40%

The asymmetry is notable: the model heavily uses deeper intra-expert reasoning but sparsely uses multi-round collaboration at this scale. A plausible interpretation is that with only four experts and 64M total parameters, per-expert refinement provides stronger marginal return than extended inter-expert message passing. This balance may shift at larger scales with stronger expert specialization.

4.4 Routing Health

From W&B diagnostics:

- router entropy remains stable and above 1.5
- drop rate remains below 5%
- expert utilization is approximately uniform (no dead experts observed)

These indicators suggest capacity masking and routing regularization are in a healthy operating regime for the tested setup.

4.5 Note on Batch Size Difference

`b-moe` used micro-batch 4 with accumulation 4, while other runs used micro-batch 8 with accumulation 2. This change was required by memory pressure from MoE parameterization. Effective batch size was still matched across comparisons (16 on RX 6600, 32 on A10), preserving fair optimization dynamics.

5 Related Work

MoC is situated at the intersection of four lines of work.

Sparse MoE scaling. Prior work established sparse expert routing as an effective scaling strategy, including Switch Transformer [6], GShard [10], Mixtral [9], and DeepSeek-MoE [4]. These methods primarily focus on routing and specialization with expert outputs merged after independent processing.

Expert communication. Communication-aware MoE proposals (including DEMoE-style approaches [1]) explore explicit cross-expert interaction, often with attention-like mechanisms whose dense all-expert forms can scale poorly. MoC addresses this concern with mediator-hub collaboration over selected experts only, yielding linear complexity in top-k experts.

Adaptive compute. Adaptive-depth and adaptive-halting ideas appear in Universal Transformer [5], PonderNet [3], and CALM [11]. MoC adopts this principle inside sparse expert layers and collaboration modules.

Iterative/implicit reasoning. Weight-shared iterative refinement connects to deep equilibrium perspectives [2]. Emerging latent-reasoning directions such as Coconut [8] and pause-token delayed-compute methods [7] further motivate allocating extra internal computation before committing outputs.

To our knowledge, MoC is among the first implementations combining sparse routing, explicit mediator collaboration, iterative expert refinement, and adaptive compute gating in one unified decoder architecture.

6 Limitations and Future Work

6.1 Limitations

This report intentionally emphasizes limitations:

- validated only at ∼64M parameters (`d_model=384`, 6 layers)

- single seed (1337), no confidence intervals
- single dataset and token budget (∼328M tokens at seq len 512)
- expert count fixed to 4, top-2 routing; 8/16/32 experts untested
- per-expert execution still uses a Python loop (no grouped GEMM path)
- evaluation limited to perplexity; no downstream task suite
- fuse gate conditions only on original input x, not on collaboration output
- micro-batch size differs for MoE vs other runs (effective batch matched)
- sequence length limited to 512; long-context behavior untested

6.2 Future Work

Priority next steps:

- scale studies at 300M, 600M, and 1B parameters
- multi-seed trials with confidence intervals
- larger expert pools (8 and 16 experts)
- grouped GEMM kernels for higher throughput
- downstream benchmarks (MMLU, HumanEval, GSM8K)
- long-context training/evaluation (2048 and 4096)
- hybrid latent + explicit reasoning designs (IRL with CoT-like supervision)
- TPU/XLA port for cross-hardware validation
- fuse-gate variants conditioned on collaboration output

7 Conclusion

At controlled small scale, MoC improves over standard MoE by enabling selected experts to collaborate through a learned mediator while preserving sparse-compute efficiency. The ablation cleanly decomposes gains: expert routing contributes the largest perplexity reduction (-12.9% vs dense baseline), mediator collaboration plus fixed IRL adds further improvement (-4.1% over MoE), and adaptive compute provides an additional -0.5% while reducing average compute usage. The implementation is open and reproducible. The key open question is scaling behavior: whether collaboration gains expand with larger models, more experts, and broader evaluation tasks.

8 Reproducibility Statement

- Code repository: https://github.com/Auren-Research/lunaris (tag `v0.1.0-beta`)
- Experiment logs: https://wandb.ai/smeryylle-moon-cloud-services-/lunaris-moc-validation
- Exact YAML configs: included in the repository for all four runs
- Hardware used: NVIDIA A10 24GB (primary), AMD RX 6600 8GB (secondary)

References

[1] Demoe, 2024. Cross-expert-attention MoE line referenced in practitioner literature; archival bibliographic metadata is not yet standardized.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1909.01377.

[3] Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021. URL https://arxiv.org/abs/2107.05407.

[4] Damai Dai, Chao Deng, Cheng Zhao, Rihuan Xu, Shihao Tang, Renrui Chen, Deli Zhao, Kai Yu, and Zhifang Sui. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024. URL https://arxiv.org/abs/2401.06066.

[5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://openreview.net/forum?id=HyzdRiR9Y7.

[6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

[7] Sachin Goyal, Krishnamurthy Dvijotham, Su Wang, Alexander Todorov, Daniel Korat, and Sarath Chandar. Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=oBpHVLzkfm.

[8] Bingbin Hao, Sainbayar Sinha, Yujin Chai, Alham Fikri Aji, Ryan Cotterell, and Abulhair Saparov. Coconut: Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL https://arxiv.org/abs/2412.06769.

[9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL https://arxiv.org/abs/2401.04088.

[10] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb.

[11] Tal Schuster, Adam Fisch, Andrew Jaegle, Jack Clark, Riki Nakajima, Ziyu Yao, Weizhu He, Shiyu Chang, Prafulla Dhariwal, Roberta Raileanu, et al. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2207.07061.